

# TP MÉTHODES NUMÉRIQUES RÉSOLUTION DE L'ÉQUATION DE POISSON

Ken Zou

Promotion 142

## Introduction

Dans le cadre de l'électrostatique, l'équation de Poisson résulte donc de la combinaison de l'équation de Maxwell-Gauss  $\text{div } \vec{E} = \rho/\varepsilon_0$  et de la définition du potentiel  $\vec{E} = -\vec{\text{grad}}V$  pour décrire le champ de potentiel  $V$  à distribution de charge définie. En notant  $\rho$  la densité de charge volumique et  $\varepsilon_0$  la permittivité électrique du vide,

$$\Delta V = -\frac{\rho}{\varepsilon_0}$$

Le but de ce projet est de **cartographier** le champ  $V(x, y)$  à 2D dans l'espace par résolution de l'équation de Poisson sur un domaine carré  $\Omega = [0, 1] \times [0, 1]$  découpé par maillage dans le plan  $Oxy$ . L'approche numérique consiste à faire une résolution par différences finies, à remplacer les dérivées partielles par des développements de Taylor, et d'utiliser plusieurs méthodes d'intégration numérique afin de comparer leur efficacité et leur complexité.

## 1 Principe numérique

### 1.1 Discrétisation

La FIGURE 1 montre la configuration d'un condensateur plan tel que  $\begin{cases} \text{charges } + : & 0.25 \leq x \leq 0.75 \text{ u.a.}, y = 0.4 \text{ u.a.} \\ \text{charges } - : & 0.25 \leq x \leq 0.75 \text{ u.a.}, y = 0.6 \text{ u.a.} \end{cases}$

Pour pouvoir résoudre numériquement cette équation différentielle, on va discrétiser les axes  $x$  et  $y$  :  $x_i = x_0 + i\Delta x$  pour  $0 \leq i < I$  et  $y_j = y_0 + j\Delta y$  pour  $0 \leq j < J$ . On prend  $I = J = N = 65$  points. Ainsi,  $\Delta x = \Delta y = 1/N$ .

Par discrétisation des dérivées partielles par **développement de Taylor à l'ordre 2** puis indexation selon les indices  $(i, j)$ , on obtient l'équation de Poisson discrétisée

$$\frac{1}{\Delta x^2}(V_{i+1,j} + V_{i-1,j} - 2V_{i,j}) + \frac{1}{\Delta y^2}(V_{i,j+1} + V_{i,j-1} - 2V_{i,j}) = \rho_{i,j}$$

On pose  $\varepsilon_0 = 1$  et  $\rho_{i,j} = 1$  pour simplifier le problème. On se ramène à la résolution d'une équation matricielle  $AX = b$ , où  $X$  est la matrice des  $V_{i,j}$ ,  $b$  la matrice des  $\rho_{i,j}$  et  $A$  la matrice de l'opérateur laplacien. On utilisera

différentes méthodes d'intégration pour en tirer  $X$ . Afin d'optimiser le temps de calcul, on peut passer des matrices 2D à 1D par réindexation  $k = iJ + j$  où  $0 \leq k < IJ$ . L'équation de Poisson devient alors :

$$\frac{1}{\Delta x^2}(V_{k+J} + V_{k-J}) + \frac{1}{\Delta y^2}(V_{k+1} + V_{k-1}) - 2\left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}\right)V_k = \rho_k$$

## 1.2 Conditions aux bords & Résolution

Pour avoir une unique solution  $V(x, y)$ , il faut imposer des conditions aux bords. On prend ici des **conditions de Dirichlet**, ie. le potentiel sera supposé nul en dehors du domaine  $\Omega$ , soit  $\forall(x, y) \in \partial\Omega, V(x, y) = 0$  par continuité.

Pour résoudre  $AX = b$ , on utilisera différentes méthodes d'intégration numérique, dont on relèvera pour chacune d'entre elle le **temps de calcul** nécessaire ainsi que l'**erreur de calcul** avec la fonction `np.linalg.norm` :  $e = \|AX - b\|$ , par rapport à 0.

## 2 Méthodes d'intégration

### 2.1 Pivot de Gauss

La méthode du **pivot de Gauss** consiste à transformer la matrice  $A$  en une **matrice échelonnée réduite** afin de résoudre le système linéaire, en **inversant** la matrice  $A$  de taille  $I \times J = N \times N$  par la fonction `np.linalg.inv`, puis de calculer  $X = A^{-1} \cdot b$ .

On relève que le temps de calcul est en moyenne de 8.6 s et que l'erreur en norme est de  $1.4 \times 10^{-24}$ .

Le pivot de Gauss est très précis mais trop élevé en complexité. En effet, le pivot de Gauss implique tout plein d'opérations élémentaires sur toutes les lignes de la matrice. On sait dans ce sens que le temps de calcul croît selon  $N^3$  avec la taille  $N \times N$  de la matrice du fait de la réduction de cette dernière car pour chaque ligne  $i$  de la matrice,

on fait  $n - i$  calculs pour  $n - i$  lignes et  $n - i$  colonnes, d'où  $\sum_{i=1}^{N-1} (n - i)^2 = O(N^3)$ , ce dont témoigne la FIGURE 2.

D'après la FIGURE 3, le premier graphe montre une carte colorée du potentiel électrostatique. Ce dernier décroît graduellement avec la distance aux deux plaques du condensateur. Le second graphe affiche les lignes de champ, qui sont logiquement plus denses entre les deux plaques. Le champ électrique, normal aux lignes de champ, est donc plus intense au sein du condensateur qu'en dehors. Le potentiel électrostatique est maximal près des charges positives et minimal près des charges négatives, ce qui est cohérent.

### 2.2 Méthode des matrices très creuses

Comme la matrice  $X$  contenant  $(65 \times 65) \times (65 \times 65) = 4225 \times 4225 = 17\,850\,625$  termes contient seulement 20 865 termes non nuls, il est préférable d'utiliser la **méthode des matrices très creuses** (*sparse matrix*), qui va permettre de réduire le temps de calcul en diminuant le stockage utilisé car **seuls les éléments non nuls seront enregistrés**, à l'aide de la fonction `spsolve` du module `scipy.sparse.linalg`.

On relève que le temps de calcul est en moyenne de 0.15 s et que l'erreur en norme est de  $6.5 \times 10^{-9}$ .

Malgré une moins grande précision, la méthode semble plus efficace en termes de complexité que le pivot de Gauss (facteur  $\times 57!$ ). En effet, si on pose  $m$  le nombre d'éléments non nuls, la complexité de la méthode est en  $O(m)$ .

D'après la FIGURE 4, les cartes obtenues sont similaires à celles obtenues par pivot de Gauss, ce qui est rassurant.

### 2.3 Méthode *solve* par décomposition LU

La fonction `linalg.solve` du module `numpy` décompose la matrice  $A = LU$  en un produit de deux matrices : une matrice  $L$  triangulaire inférieure et une matrice  $U$  triangulaire supérieure avec 
$$\begin{cases} \forall i > k & l_{ik} = \frac{a_{ik}}{u_{kk}} \\ \forall i > k, \forall j \geq k, & u_{ij} = u_{ij} - l_{ik}u_{kj} \end{cases}.$$

On appelle cette factorisation la **factorisation Lower-Upper (LU)**, de complexité globale en  $O(N^3)$  car dû au calcul des  $u_{ij}$ , nécessitant pour chaque  $k$  la mise à jour de  $n - k$  lignes et  $n - k$  colonnes donc  $\sum_{k=1}^{N-1} (n - k)^2 = O(N^3)$ .

On se ramène ensuite à deux systèmes triangulaires facilement résolubles, en  $O(N^2)$  :  $\begin{cases} LX' = b \\ UX = X' \end{cases}$ . La fonction `solve` résout le premier système puis le deuxième et renvoie  $X$ . Elle a une complexité globale en  $O(N^3)$ .

On relève que le temps de calcul est en moyenne de 2.6 s et que l'erreur en norme est de  $6.5 \times 10^{-9}$ .

La précision de la méthode est la même que celle des matrices creuses mais on perd en temps de calcul d'un facteur  $\times 14$ . D'après la FIGURE 5, la cartographie est conforme à ce qui est attendu.

### 2.4 Méthode de Jacobi

Dans certaines situations, il peut être intéressant d'utiliser des **méthodes itératives** car elles peuvent être significativement plus rapides en temps d'exécution, plus stables et éventuellement nécessiter beaucoup moins de mémoire de stockage. Le principe est de remplacer  $AX = b$  par une équation alternative de la forme  $X = BX + c$  où  $B$  est une matrice inversible et  $c$  un vecteur à déterminer. Le principe de l'itération consiste en, à partir d'une solution initiale  $X_0$ , d'itérer la relation de récurrence  $X_{k+1} = BX_k + c$  jusqu'à convergence de la suite  $(X_k)_{k \in \mathbb{N}}$ . On transforme donc le problème d'inversion de système linéaire en recherche de point fixe. Deux critères d'arrêt sont imposés :

- la tolérance  $\varepsilon$  telle que  $\|X_{k+1} - X_k\| < \varepsilon \|X_k\|$  ;
- un nombre maximum d'itération  $N_{iter}$  pour éviter la divergence.

Soit  $K$  une matrice inversible. En posant  $B = I - K^{-1}A$  et  $c = K^{-1}b$ , on a bien l'équivalence  $AX = b \Leftrightarrow X = BX + c$ . On peut enfin décomposer la matrice  $A = L + D + U$  où  $L$ ,  $D$ ,  $U$  sont respectivement les matrices triangulaire supérieure stricte, diagonale et triangulaire inférieure stricte.

La **méthode de Jacobi** suppose que la **matrice diagonale  $D$  soit inversible**, ie. elle ne contient aucun terme diagonal nul. Posons  $K = D$ . Alors :

$$X_{k+1} = (I - D^{-1}A)X_k + D^{-1}b = (I - D^{-1}(L + D + U))X_k + D^{-1}b = -D^{-1}[(L + U)X_k - b]$$

Composante par composante, l'équation donne  $\forall i \in \llbracket 1, I \rrbracket$ ,  $a_{ii} \neq 0$  :

$$x_{i,k+1} = -\frac{1}{a_{ii}} \left( \sum_{j \neq i} a_{ij}x_{j,k} - b_i \right)$$

On comprend d'après la formule que la méthode de Jacobi est convergente si la matrice  $A$  est à diagonale dominante ou symétrique définie positive, soit  $\forall i \in \llbracket 1, I \rrbracket$ ,  $|a_{ii}| > \sum_{j > i} |a_{ij}|$ . Pour choisir le critère de convergence  $\varepsilon$  ainsi que le nombre d'itérations maximal  $N_{iter}$ , on évalue l'efficacité de la méthode d'intégration de Jacobi.

D'après la FIGURE 6, l'erreur devient stable pour  $\varepsilon < 10^{-17}$  et vaut environ  $6.5 \times 10^{-9}$ . Donc on aurait tendance à prendre  $\varepsilon$  le plus petit possible. Mais la complexité croît selon  $N^2$ . En effet, pour calculer  $x_{i,k+1}$ , il faut  $N - 1$  opérations donc une complexité en  $O(N)$ . Comme il y a  $N$  composantes à calculer, on obtient donc une complexité totale en  $O(N^2)$ .

La complexité globale de la méthode de Jacobi est donc en  $O(kN^2)$ , où  $k$  correspond au nombre d'itérations qui dépend notamment du rayon spectral de la matrice d'itération  $-D(L+U)$ .

On prend donc comme compromis  $\varepsilon = 10^{-17}$  et  $N_{iter} = 3000$  pour toute la suite. On relève que le temps de calcul est en moyenne de 14.4 s au bout de 251 itérations pour une tolérance  $\varepsilon = 10^{-17}$ , et que l'erreur en norme par la méthode de Jacobi est de  $6.5 \times 10^{-9}$ .

La précision reste sensiblement la même, pour un temps de calcul plus élevé que les méthodes directes. D'après la FIGURE 7, les lignes de champ se sont légèrement appauvries pour des positions assez éloignées des deux plaques de charge, mais ces lignes restent d'amplitude faible devant celle des lignes dans le condensateur.

## 2.5 Méthode de Gauss-Seidel

Cette méthode suppose aussi que **la matrice diagonale  $D$  soit inversible**, ie. elle ne contient aucun terme diagonal nul, mais on pose cette fois-ci  $K = L + D$ . Alors :

$$X_{k+1} = (I - (L + D)^{-1}A)X_k + (L + D)^{-1}b = -D^{-1}[LX_{k+1} + UX_k - b]$$

Composante par composante, l'équation donne :

$$x_{i,k+1} = -\frac{1}{a_{ii}} \left( \sum_{j<i} a_{ij}x_{j,k+1} + \sum_{j>i} a_{ij}x_{j,k} - b_i \right)$$

Avant toute implémentation, on voit que la méthode de Jacobi part de l'itération suivante  $k+1$ . Dans le cas de la méthode de Gauss-Seidel, l'itération  $k+1$  a besoin de l'itération précédente  $k$ , mais également de l'itération courante  $k+1$ . On peut en déduire que :

- en cas de convergence des itérations, Gauss-Seidel devrait converger plus rapidement que Jacobi, puisque l'itération fait intervenir les itérations  $k$  et  $k+1$  ;
- en revanche en cas de divergence, Gauss-Seidel devrait diverger également plus rapidement que Jacobi.

On remarque aussi d'après la FIGURE 8 que le temps de calcul pour Gauss-Seidel est plus faible pour un critère de convergence de plus en plus fin, et de même pour le nombre d'itérations. En effet, le nombre d'itérations pour Gauss-Seidel est plus faible en complexité que celui de Jacobi car la matrice d'itération est  $-(D+L)^{-1}U$  alors que pour Jacobi c'était  $-D^{-1}(L+U)$ . La complexité totale de la méthode de Gauss-Seidel est donc en  $O(k_{GS}N^2)$

On relève que le temps de calcul par la méthode de Gauss-Seidel est de 13.2 s au bout de 226 itérations pour une tolérance de  $10^{-17}$ , et que l'erreur en norme par la méthode de Gauss-Seidel est de  $6.5 \times 10^{-9}$ .

On retrouve bien que le nombre d'itérations pour Gauss-Seidel est plus faible pour converger que celui de Jacobi. D'après la FIGURE 9, la cartographie est cohérente.

## 2.6 Méthode de Gauss-Seidel avec sur-relaxation

On reprend la méthode de Gauss-Seidel en introduisant **un facteur de relaxation  $\omega$**  qu'on va choisir de façon optimale afin de minimiser l'erreur et la complexité.

$$x_{i,k+1} = (1 - \omega)x_k + \frac{\omega}{a_{ii}} \left( b_i - \sum_{j<i} a_{ij}x_{j,k+1} - \sum_{j>i} a_{ij}x_{j,k} \right)$$

- Pour  $0 < \omega < 1$ , on parle de sous-relaxation, ce qui permet d'améliorer la stabilité.
- Pour  $\omega > 1$ , on parle de sur-relaxation, ce qui permet d'améliorer la convergence.

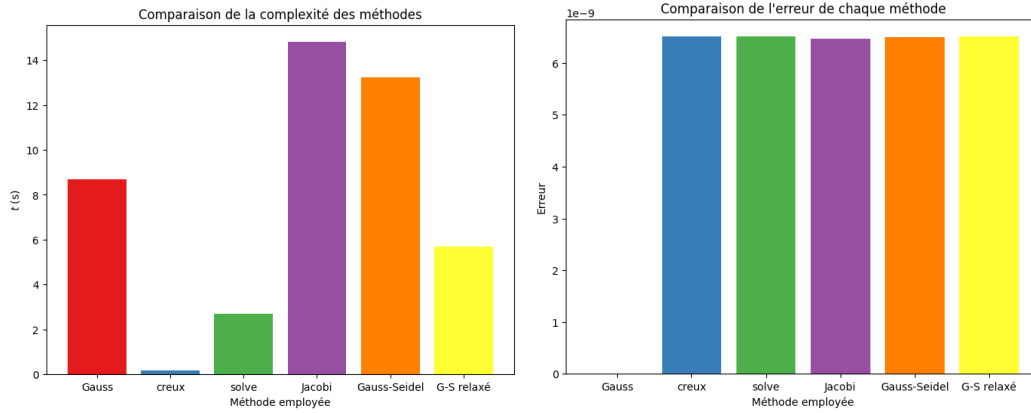
Le facteur de relaxation optimal est celui qui minimise le temps de calcul ainsi que l'erreur de calcul. On pourrait dire d'après la FIGURE 10 que  $\omega = 0$  semble être un bon choix, sauf que cela signifie que  $x_{i,k+1} = x_k$ , le potentiel est donc constant dans l'espace, ce qui n'est pas physiquement acceptable. Globalement, pour  $\omega < 0.5$ , le champ obtenu n'est pas satisfaisant malgré un temps de calcul réduit. On choisit donc plutôt  $\omega = 1.9$ , ce qui donne un champ plutôt satisfaisant d'après la FIGURE 11 et la FIGURE 12.

On relève que pour  $\omega = 1.9$ , le temps de calcul par la méthode de Gauss-Seidel avec relaxation est de 5.7 s au bout de 82 itérations pour une tolérance de  $10^{-17}$ , et l'erreur en norme par la méthode de Gauss-Seidel avec relaxation est de  $6.5 \times 10^{-9}$ .

On gagne un facteur  $\times 3$  sur le nombre d'itérations et un facteur  $\times 2.5$  sur la complexité, ce qui montre l'intérêt d'avoir un facteur de relaxation.

## Conclusion

Au final, la méthode la plus optimale en termes de complexité est la méthode des matrices creuses. La méthode la plus optimale sur la précision est le pivot de Gauss. On a aussi remarqué durant ce projet que les méthodes itératives sont beaucoup plus stables pour résoudre des systèmes linéaires avec un très grand nombre d'équations.



Pour  $k_J > k_{GS} > k_{SOR}$ , on peut résumer les complexités dans le tableau suivant :

| Méthode    | Pivot de Gauss | Matrices creuses | Solve    | Jacobi       | Gauss-Seidel    | Gauss-Seidel SOR |
|------------|----------------|------------------|----------|--------------|-----------------|------------------|
| Complexité | $O(N^3)$       | $O(m)$           | $O(N^3)$ | $O(k_J N^2)$ | $O(k_{GS} N^2)$ | $O(k_{SOR} N^2)$ |

On peut aussi comparer l'allure du potentiel trouvé dans les 3 méthodes d'intégration directe et les 3 méthodes d'intégration itérative par différence des potentiels solutions calculées  $|X_i - X_j|$ , cf. la FIGURE 13 et la FIGURE 14.

On conclut que la résolution de l'équation de Poisson peut se faire aisément numériquement, avec des méthodes d'intégration diverses et variées. Pour cela, il a fallu discrétiser l'équation pour se ramener à un système d'équations linéaires à résoudre.

## Annexe avec toutes les figures obtenues numériquement

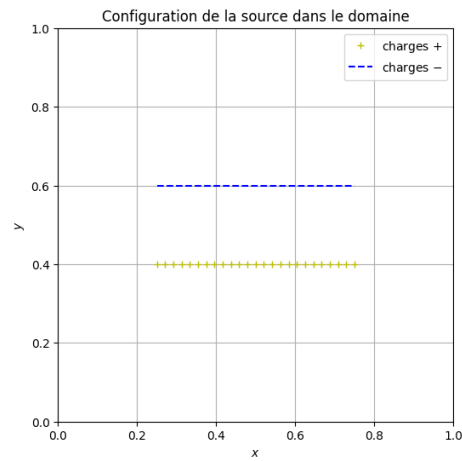


FIGURE 1 – Configuration du condensateur plan

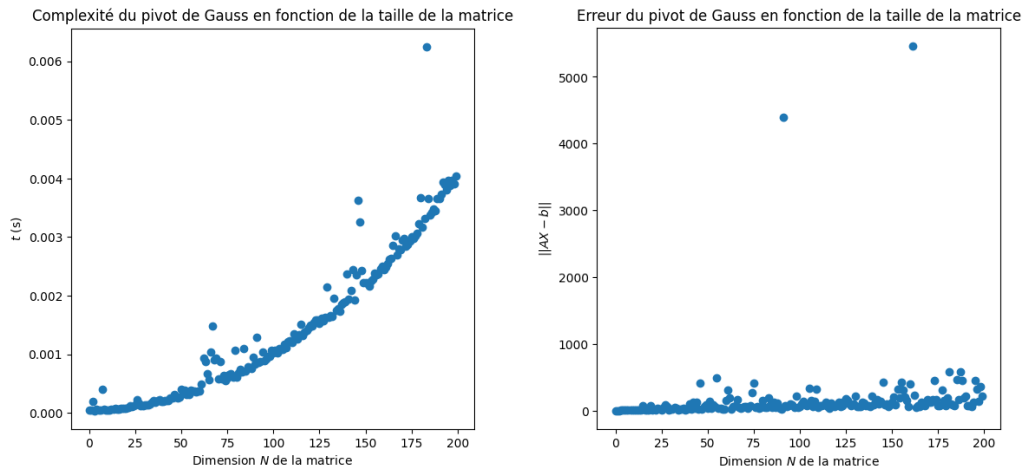


FIGURE 2 – Évolution du temps de calcul du pivot de Gauss et de son erreur

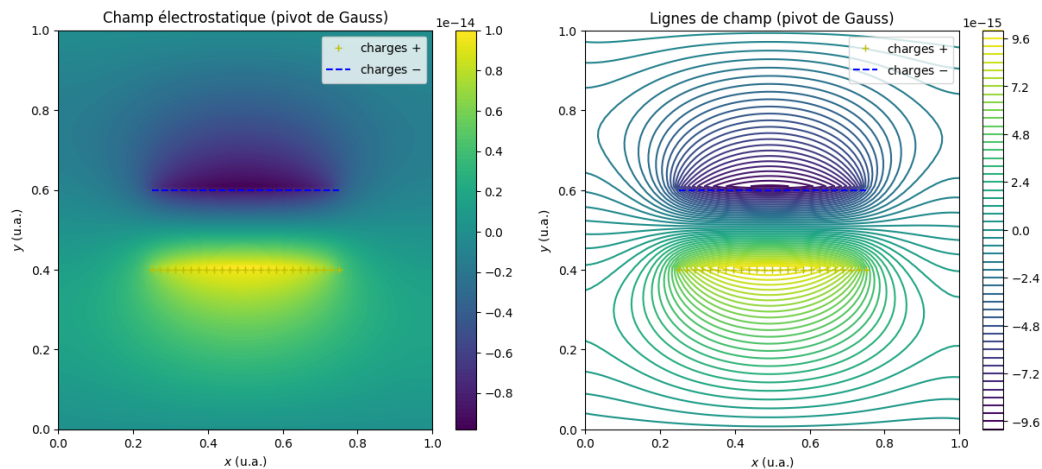


FIGURE 3 – Champ électrostatique obtenu par pivot de Gauss

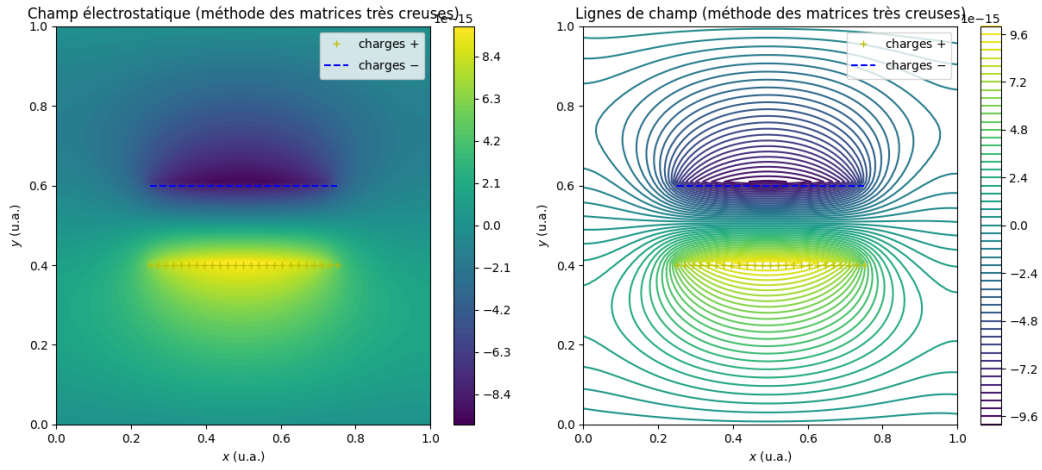


FIGURE 4 – Champ électrostatique obtenu par méthode des matrices très creuses

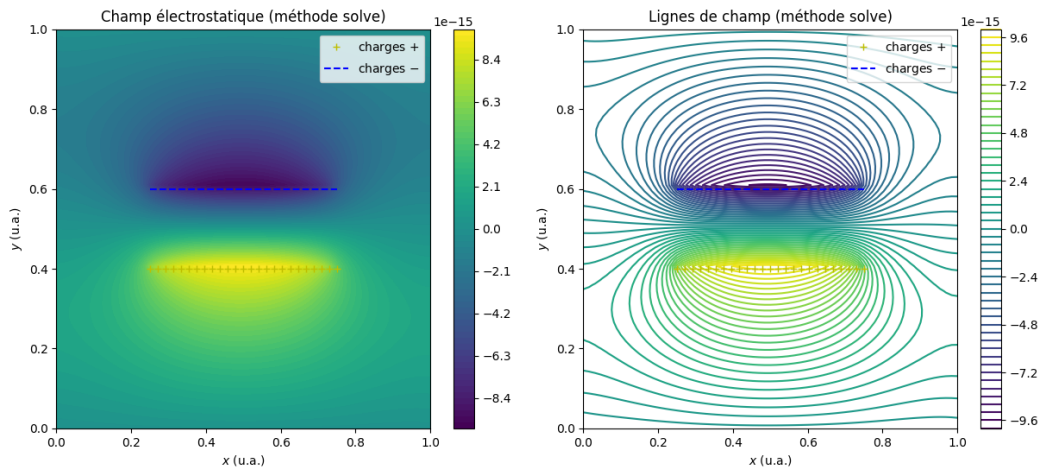
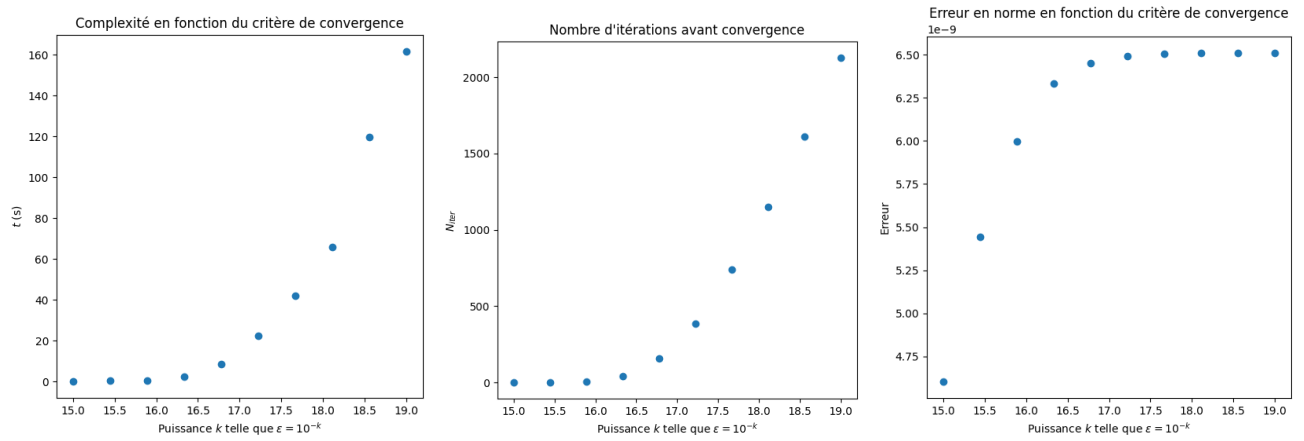
FIGURE 5 – Champ électrostatique obtenu par méthode *solve*

FIGURE 6 – Évolution du temps de calcul, du nombre d'itération et de l'erreur de la méthode de Jacobi

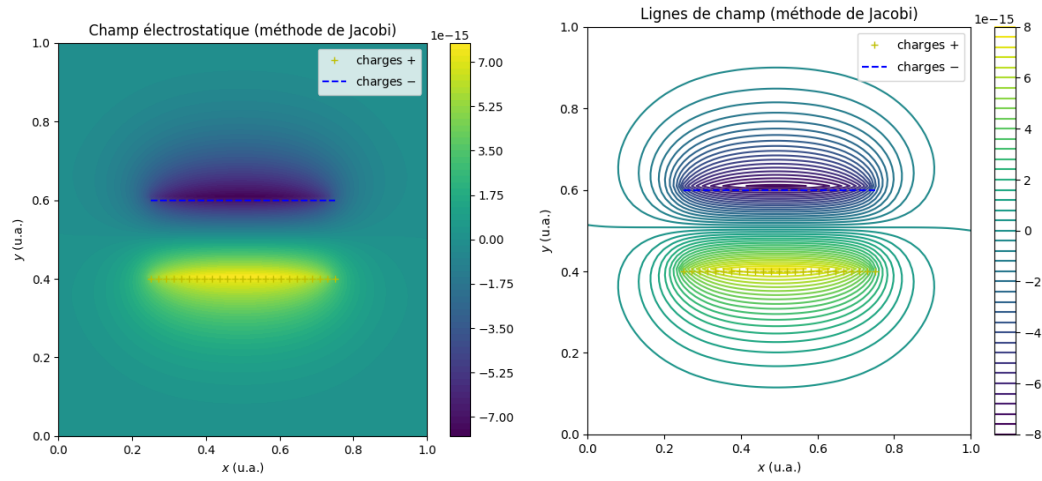


FIGURE 7 – Champ électrostatique obtenu par méthode de Jacobi

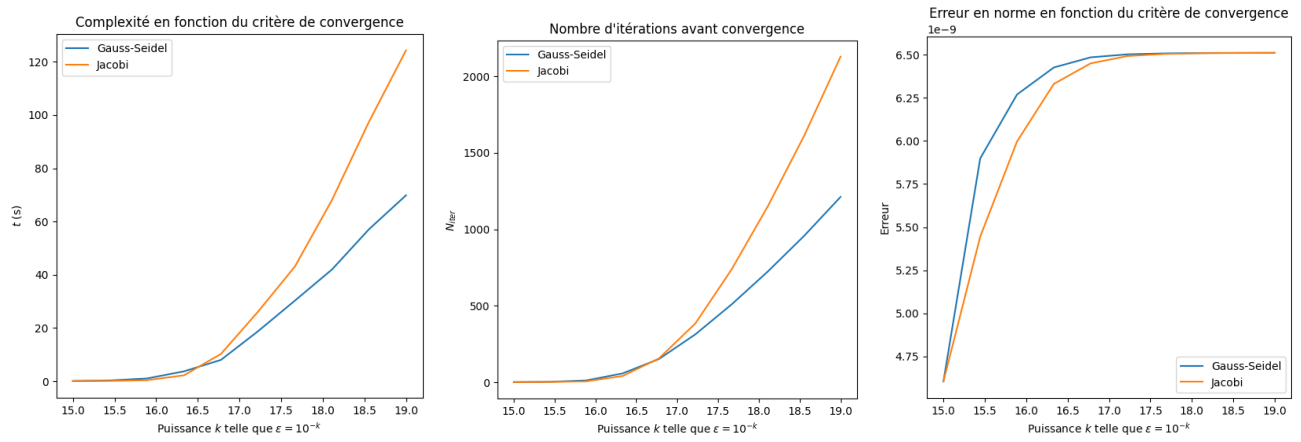


FIGURE 8 – Comparaison des efficacités de Jacobi et Gauss-Seidel

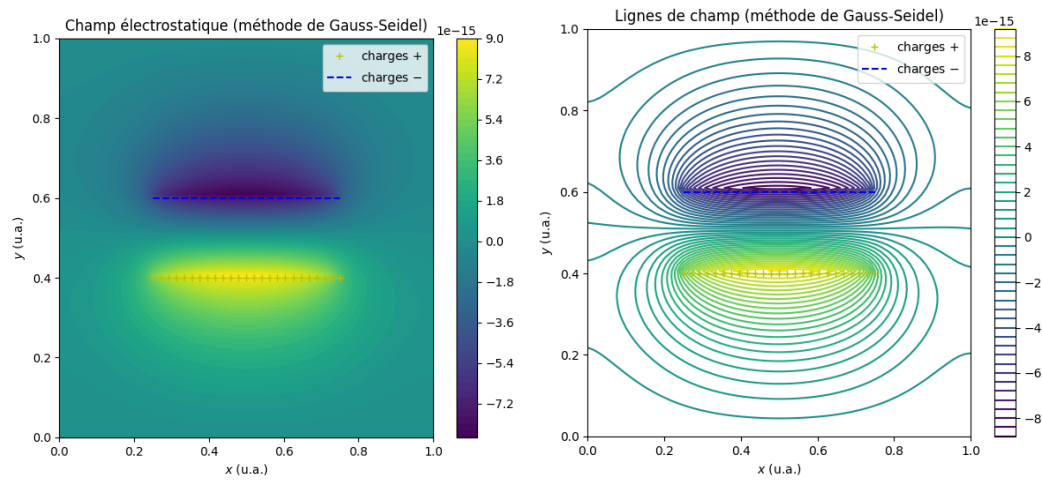


FIGURE 9 – Champ électrostatique obtenu par méthode de Gauss-Seidel

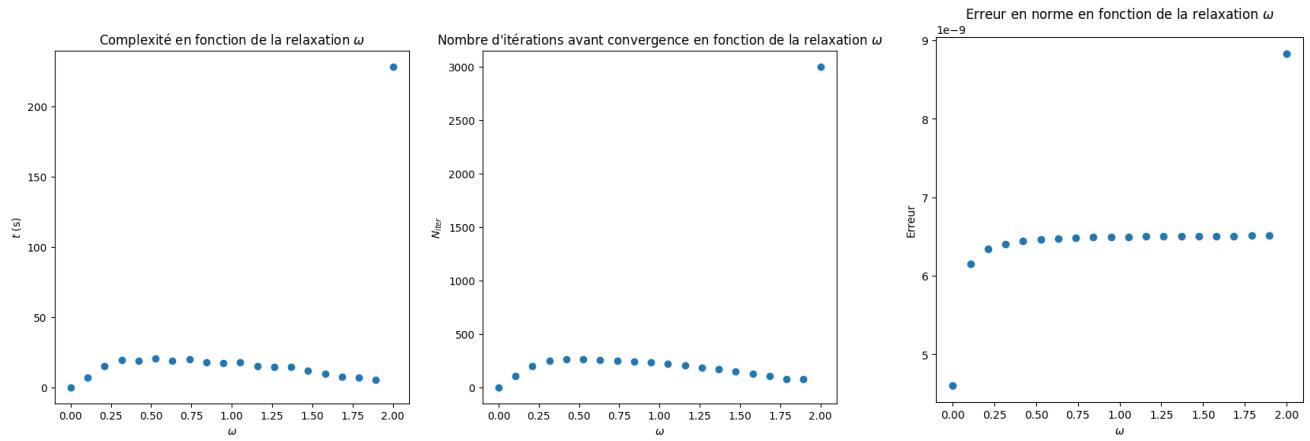


FIGURE 10 – Évolution de l'efficacité en fonction du facteur de relaxation

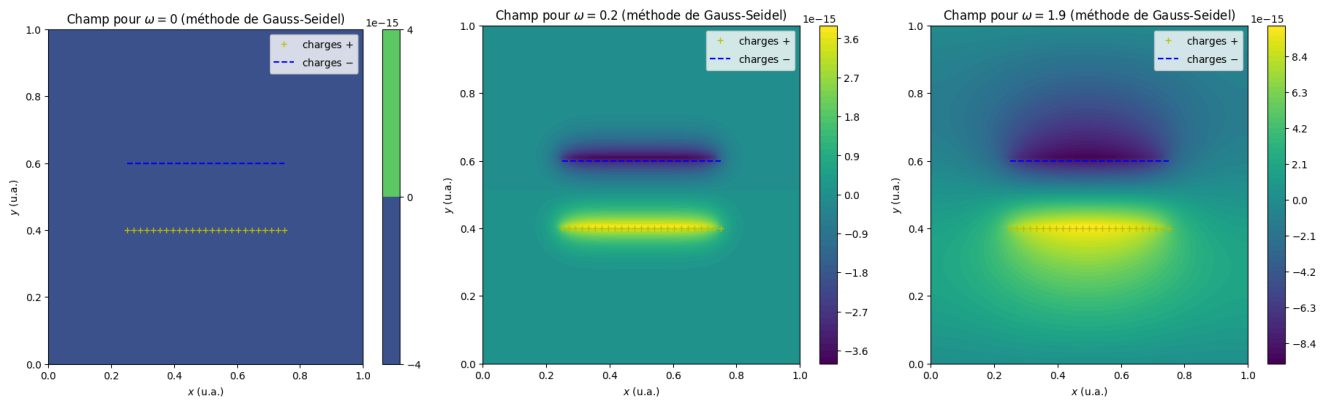
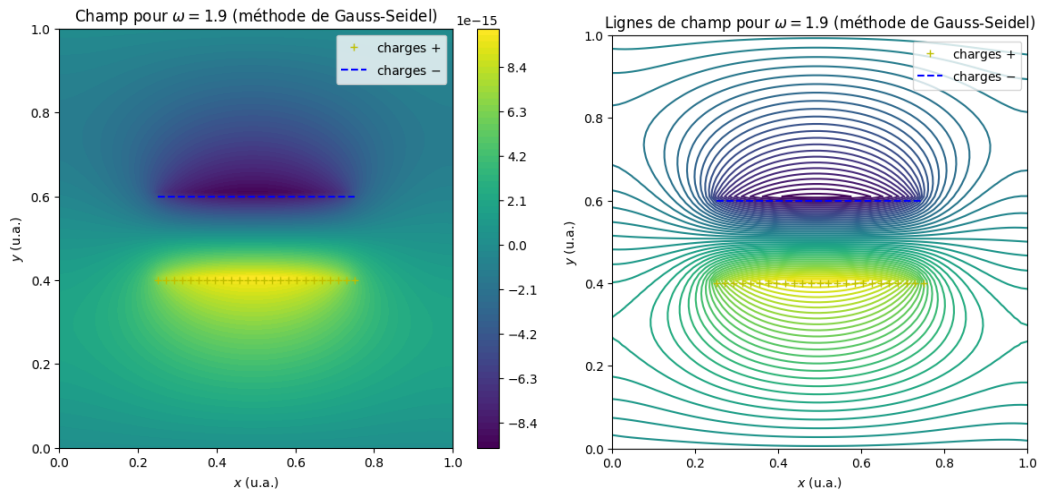
FIGURE 11 – Comparaison des champs en fonction du facteur  $\omega$ 

FIGURE 12 – Champ électrostatique obtenu par méthode de Gauss-Seidel

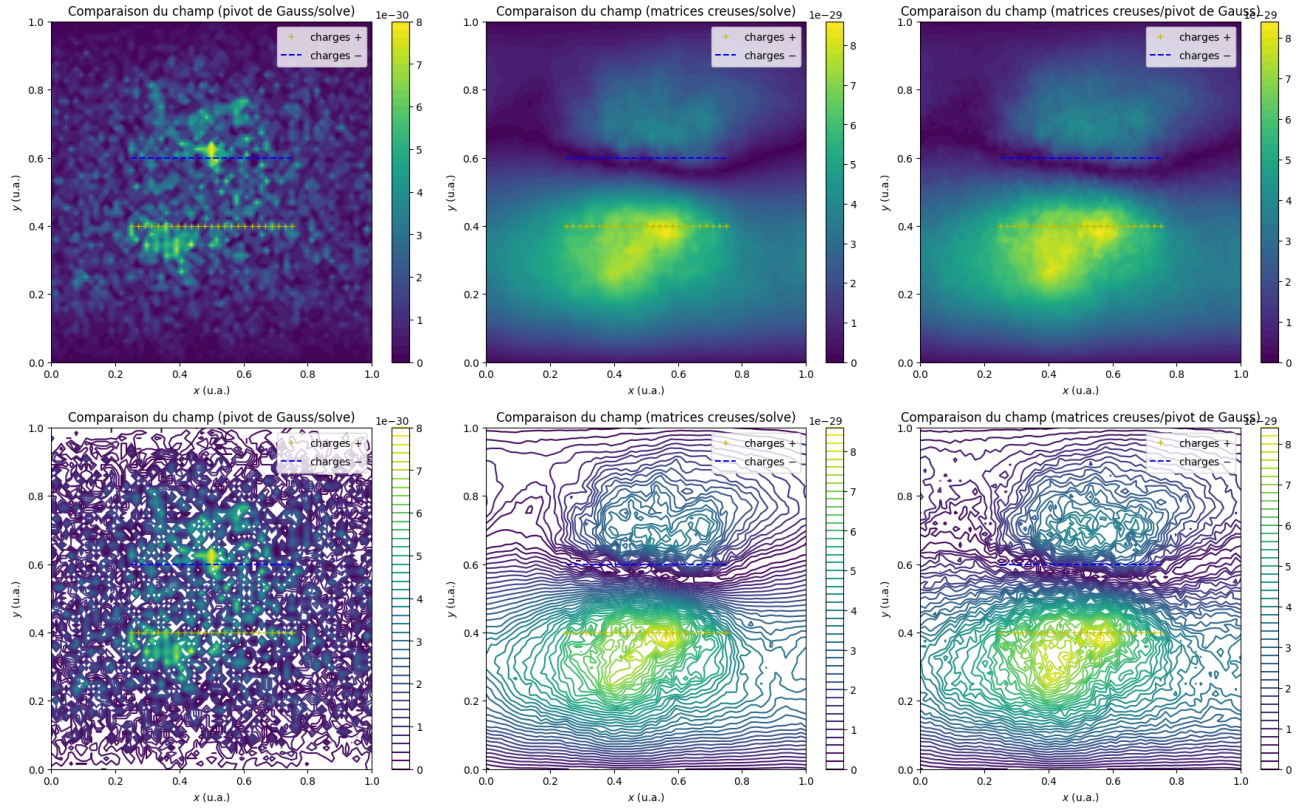


FIGURE 13 – Comparaison du champ des 3 méthodes directes

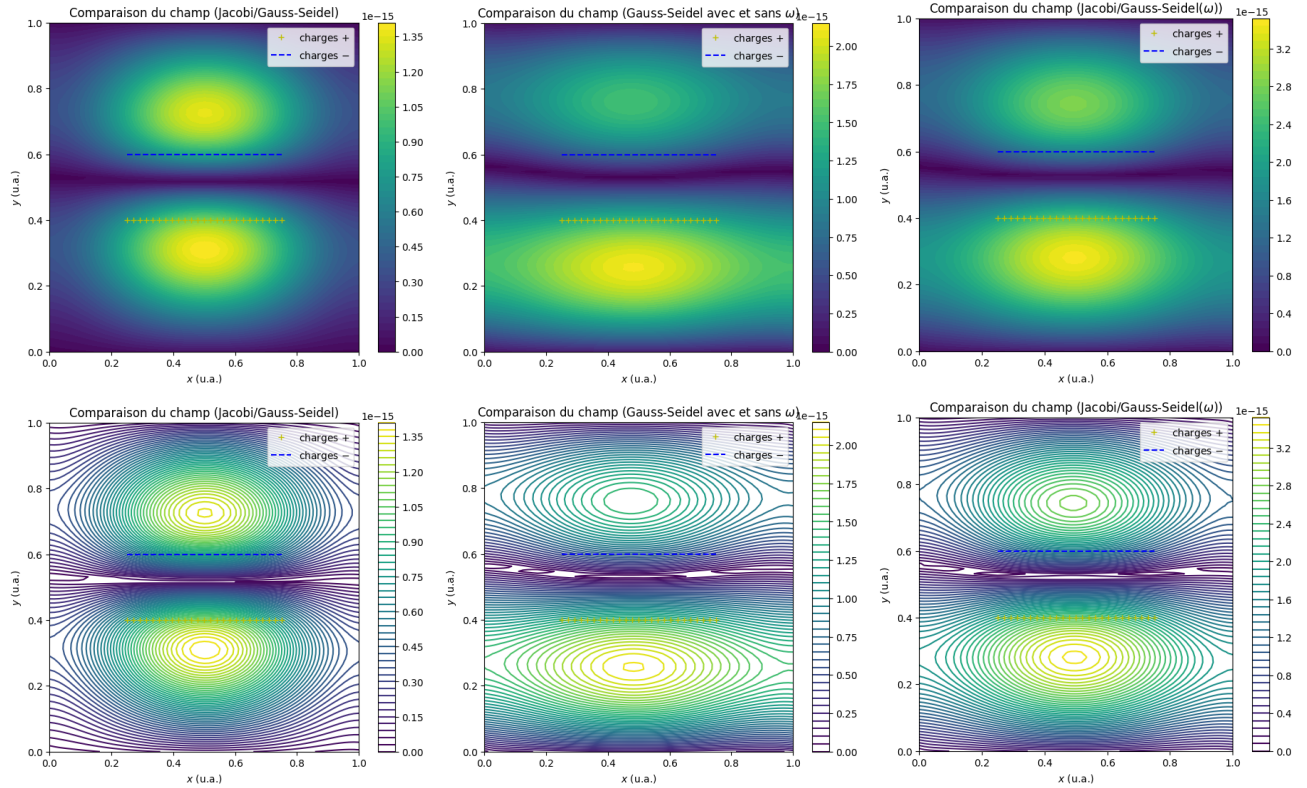


FIGURE 14 – Comparaison du champ des 3 méthodes itératives